Cosmos EVM Architecture: Complete Analysis

목차

- 개요
- Cosmos EVM이란?
- 전체 아키텍처
- x/evm 모듈 구조
- StateDB 구현
- Transaction Processing
- Account 모델
- Storage Management
- Gas Mechanism
- Precompiled Contracts
- 실전 코드 분석
- 참고 자료

개요

Cosmos EVM은 Ethereum Virtual Machine을 Cosmos SDK 블록체인에 통합하는 프로젝트입니다. 이를 통해 Cosmos 생태계에서 Ethereum 스마트 컨트랙트를 실행할 수 있습니다.

학습 목표:

- Cosmos EVM의 전체 아키텍처 이해
- StateDB와 Cosmos SDK KVStore 통합 방식
- EVM 트랜잭션 처리 플로우
- Account 모델의 차이점 이해
- Gas 메커니즘 통합

예상 학습 시간: 4-5시간

Cosmos EVM이란?

핵심 개념

Cosmos EVM은 **EVM 실행 환경**을 **Cosmos SDK 상태 관리** 위에 구축합니다.

```
┌─────────────────────────────────────────┐
│        Ethereum Compatible Layer        │
│  - EVM bytecode execution               │
│  - Solidity smart contracts             │
│  - Web3 JSON-RPC API                    │
└─────────────────────────────────────────┘
                  │
                  │
                  ▼
┌─────────────────────────────────────────┐
│            x/evm Module                 │
│  - StateDB (EVM state → Cosmos state)   │
│  - MsgEthereumTx handler                │
│  - EVM hooks                            │
└─────────────────────────────────────────┘
                  │
                  │
                  ▼
┌─────────────────────────────────────────┐
│        Cosmos SDK (State Layer)         │
│  - KVStore, IAVL tree                   │
│  - Transaction processing               │
│  - Consensus (CometBFT)                 │
└─────────────────────────────────────────┘
```

주요 구성 요소

1. **x/evm 모듈**: EVM 실행 로직
2. **StateDB**: EVM state ↔ Cosmos SDK KVStore 브릿지
3. **MsgEthereumTx**: Ethereum 트랜잭션 래퍼
4. **JSON-RPC Server**: Ethereum 호환 API
5. **Precompiles**: Cosmos SDK 기능을 EVM에 노출

전체 아키텍처

레이어 구조

```
┌─────────────────────────────────────────────────────────┐
│                    Client Layer                         │
│  - MetaMask, Web3.js, ethers.js                         │
│  - JSON-RPC calls (eth_sendTransaction, eth_call, etc.) │
└─────────────────────────────────────────────────────────┘
                          │
                          ▼
┌─────────────────────────────────────────────────────────┐
│                   JSON-RPC Server                       │
│  - Translates Ethereum RPC to Cosmos queries/txs        │
│  - eth_* namespace → Cosmos SDK queries                 │
└─────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────┐
│                  x/evm Module                        │
│                                                      │
│  ┌──────────────────────────────────────────────┐    │
│  │           MsgEthereumTx Handler              │    │
│  │  - Unwrap Ethereum transaction               │    │
│  │  - Verify signature (ECDSA)                  │    │
│  │  - Execute EVM bytecode                      │    │
│  └──────────────────────────────────────────────┘    │
│                    │                                 │
│  ┌─────────────────▼────────────────────────────┐    │
│  │              EVM Executor                    │    │
│  │  - Create EVM instance                       │    │
│  │  - Call/Create contract                      │    │
│  │  - OpCode execution                          │    │
│  └──────────────────────────────────────────────┘    │
│                    │                                 │
│  ┌─────────────────▼────────────────────────────┐    │
│  │              StateDB                         │    │
│  │  - EVM state interface                       │    │
│  │  - Account: nonce, balance, code, storage    │    │
│  │  - Maps to Cosmos SDK KVStore                │    │
│  └──────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────┐
│              Cosmos SDK KVStore                      │
│  ┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐  │
│  │Account  │  │ Balance │  │  Code   │  │ Storage │  │
│  │ Store   │  │  Store  │  │  Store  │  │  Store  │  │
│  └─────────┘  └─────────┘  └─────────┘  └─────────┘  │
└──────────────────────────────────────────────────────┘
```

x/evm 모듈 구조

디렉터리 구조

```
x/evm/
├── keeper/
│   ├── keeper.go          # Keeper 구조체
│   ├── statedb.go         # StateDB 구현
│   ├── msg_server.go      # MsgEthereumTx 핸들러
│   └── grpc_query.go      # gRPC 쿼리 핸들러
├── types/
│   ├── tx.pb.go           # MsgEthereumTx 정의
│   ├── state_object.go    # StateObject (계정 상태)
│   ├── statedb.go         # StateDB 인터페이스
│   └── journal.go         # State revert 메커니즘
├── vm/
│   └── interface.go       # EVM 인터페이스
└── module.go              # AppModule 구현
```

Keeper 구조

```go
// keeper/keeper.go
type Keeper struct {
    storeKey      sdk.StoreKey
    cdc           codec.BinaryCodec

    // Account keeper for account management
    accountKeeper types.AccountKeeper

    // Bank keeper for token transfers
    bankKeeper    types.BankKeeper

    // Staking keeper for validator queries
    stakingKeeper types.StakingKeeper

    // Fee collector module account
    feeCollector  string

    // EVM tracer for debugging
    tracer        string
}

func NewKeeper(
    cdc codec.BinaryCodec,
    storeKey sdk.StoreKey,
    ak types.AccountKeeper,
    bk types.BankKeeper,
    sk types.StakingKeeper,
    feeCollector string,
) *Keeper {
    return &Keeper{
        storeKey:      storeKey,
        cdc:           cdc,
        accountKeeper: ak,
        bankKeeper:    bk,
        stakingKeeper: sk,
        feeCollector:  feeCollector,
    }
}
```

StateDB 구현

StateDB 인터페이스

StateDB는 EVM이 요구하는 상태 인터페이스를 Cosmos SDK KVStore 위에 구현합니다.

```go
// types/statedb.go
type StateDB struct {
    keeper        Keeper
    ctx           sdk.Context

    // State objects (accounts)
    stateObjects      map[common.Address]*StateObject
    stateObjectsDirty map[common.Address]struct{}

    // Journal for reverting changes
    journal        *journal
    validRevisions []revision
    nextRevisionId int

    // Precompiled contracts
    precompiles map[common.Address]vm.PrecompiledContract

    // Logs
    logs    map[common.Hash][]*ethtypes.Log
    logSize uint

    // Refund counter
    refund uint64
}
```

StateObject (Account State)

```go
// types/state_object.go
type StateObject struct {
    db      *StateDB
    address common.Address

    // Account state
    account Account

    // Storage cache
    originStorage Storage  // Original storage (for revert)
    dirtyStorage  Storage  // Modified storage

    // Flags
    dirtyCode bool
    suicided  bool
}

type Account struct {
    Nonce    uint64
    Balance  *big.Int
    CodeHash []byte
}

type Storage map[common.Hash]common.Hash
```

Key 설계

EVM 상태를 Cosmos SDK KVStore에 매핑하는 key 구조:

```go
// types/keys.go
const (
    // Prefix for different stores
    KeyPrefixAccount  = 0x01  // Account state
    KeyPrefixCode     = 0x02  // Contract code
    KeyPrefixStorage  = 0x03  // Contract storage
)

// Account key: prefix + address
func AccountKey(addr common.Address) []byte {
    return append([]byte{KeyPrefixAccount}, addr.Bytes()...)
}

// Code key: prefix + code hash
func CodeKey(codeHash []byte) []byte {
    return append([]byte{KeyPrefixCode}, codeHash...)
}

// Storage key: prefix + address + storage key
func StorageKey(addr common.Address, key common.Hash) []byte {
    prefix := []byte{KeyPrefixStorage}
    prefix = append(prefix, addr.Bytes()...)
    prefix = append(prefix, key.Bytes()...)
    return prefix
}
```

StateDB 주요 메소드

Account 관리

```go
// Get account
func (s *StateDB) GetAccount(addr common.Address) *StateObject {
    // Check cache first
    if obj := s.stateObjects[addr]; obj != nil {
        return obj
    }

    // Load from store
    store := s.ctx.KVStore(s.keeper.storeKey)
    bz := store.Get(AccountKey(addr))

    if bz == nil {
        // Account doesn't exist, create empty one
        return s.createObject(addr)
    }

    var account Account
    s.keeper.cdc.MustUnmarshal(bz, &account)

    obj := newStateObject(s, addr, account)
    s.stateObjects[addr] = obj
    return obj
}

// Create account
func (s *StateDB) CreateAccount(addr common.Address) {
    obj := s.GetAccount(addr)
    obj.account = Account{
        Nonce:    0,
        Balance:  big.NewInt(0),
        CodeHash: emptyCodeHash,
    }
    s.journal.append(createObjectChange{account: &addr})
}
```

Balance 관리

```go
// Get balance
func (s *StateDB) GetBalance(addr common.Address) *big.Int {
    obj := s.GetAccount(addr)
    if obj != nil {
        return obj.account.Balance
    }
    return big.NewInt(0)
}

// Add balance
func (s *StateDB) AddBalance(addr common.Address, amount *big.Int) {
    obj := s.GetOrNewStateObject(addr)
    if obj != nil {
        obj.AddBalance(amount)
    }
}

// Sub balance
func (s *StateDB) SubBalance(addr common.Address, amount *big.Int) {
    obj := s.GetOrNewStateObject(addr)
    if obj != nil {
        obj.SubBalance(amount)
    }
}
```

Storage 관리

```go
// Get storage
func (s *StateDB) GetState(addr common.Address, key common.Hash) common.Hash {
    obj := s.GetAccount(addr)
    if obj != nil {
        return obj.GetState(key)
    }
    return common.Hash{}
}

// Set storage
func (s *StateDB) SetState(addr common.Address, key, value common.Hash) {
    obj := s.GetOrNewStateObject(addr)
    if obj != nil {
        obj.SetState(key, value)
    }
}

// StateObject.GetState
func (so *StateObject) GetState(key common.Hash) common.Hash {
    // Check dirty storage first
    if value, ok := so.dirtyStorage[key]; ok {
        return value
    }

    // Check origin storage
    if value, ok := so.originStorage[key]; ok {
        return value
    }

    // Load from database
    store := so.db.ctx.KVStore(so.db.keeper.storeKey)
    bz := store.Get(StorageKey(so.address, key))

    if bz == nil {
        return common.Hash{}
    }

    var value common.Hash
    copy(value[:], bz)
    so.originStorage[key] = value
    return value
}
```

Code 관리

```go
// Get code
func (s *StateDB) GetCode(addr common.Address) []byte {
    obj := s.GetAccount(addr)
    if obj != nil {
        return obj.Code()
    }
    return nil
}

// Set code
func (s *StateDB) SetCode(addr common.Address, code []byte) {
    obj := s.GetOrNewStateObject(addr)
    if obj != nil {
        obj.SetCode(crypto.Keccak256Hash(code), code)
    }
}

// StateObject.Code
func (so *StateObject) Code() []byte {
    if so.code != nil {
        return so.code
    }

    if bytes.Equal(so.account.CodeHash, emptyCodeHash) {
        return nil
    }

    // Load from database
    store := so.db.ctx.KVStore(so.db.keeper.storeKey)
    code := store.Get(CodeKey(so.account.CodeHash))
    so.code = code
    return code
}
```

Transaction Processing

MsgEthereumTx 구조

```go
// types/tx.pb.go
type MsgEthereumTx struct {
    // Inner transaction data
    Data *types.Any `protobuf:"bytes,1,opt,name=data,proto3" json:"data,omitempty"`

    // Deprecated fields (for backwards compatibility)
    Size_ float64 `protobuf:"fixed64,2,opt,name=size,proto3" json:"-"`
    Hash  string  `protobuf:"bytes,3,opt,name=hash,proto3" json:"hash,omitempty"`
    From  string  `protobuf:"bytes,4,opt,name=from,proto3" json:"from,omitempty"`
}

// Inner transaction types
type LegacyTx struct {
    Nonce    uint64
    GasPrice *big.Int
    Gas      uint64
    To       *common.Address
    Value    *big.Int
    Data     []byte
    V, R, S  *big.Int  // Signature
}

type AccessListTx struct {
    ChainID    *big.Int
    Nonce      uint64
    GasPrice   *big.Int
    Gas        uint64
    To         *common.Address
    Value      *big.Int
    Data       []byte
    AccessList AccessList
    V, R, S    *big.Int
}

type DynamicFeeTx struct {
    ChainID    *big.Int
    Nonce      uint64
    GasTipCap  *big.Int  // EIP-1559
    GasFeeCap  *big.Int  // EIP-1559
    Gas        uint64
    To         *common.Address
    Value      *big.Int
    Data       []byte
    AccessList AccessList
    V, R, S    *big.Int
}
```

Transaction 실행 플로우

```go
// keeper/msg_server.go
func (k *Keeper) EthereumTx(
    goCtx context.Context,
    msg *types.MsgEthereumTx,
) (*types.MsgEthereumTxResponse, error) {
    ctx := sdk.UnwrapSDKContext(goCtx)

    // 1. Unpack inner transaction
    tx := msg.AsTransaction()

    // 2. Verify signature
    sender, err := tx.Sender()
    if err != nil {
        return nil, err
    }

    // 3. Create StateDB
    stateDB := NewStateDB(ctx, k)

    // 4. Apply transaction
    result, err := k.ApplyTransaction(ctx, stateDB, tx)
    if err != nil {
        return nil, err
    }

    // 5. Commit state changes
    if err := stateDB.Commit(); err != nil {
        return nil, err
    }

    // 6. Return result
    return &types.MsgEthereumTxResponse{
        Hash: tx.Hash().Hex(),
        Logs: result.Logs,
        Ret:  result.Ret,
    }, nil
}
```

ApplyTransaction

```go
func (k *Keeper) ApplyTransaction(
    ctx sdk.Context,
    stateDB *StateDB,
    tx *ethtypes.Transaction,
) (*ExecutionResult, error) {
    // 1. Prepare EVM config
    cfg := k.GetEVMConfig(ctx)

    // 2. Create EVM instance
    evm := vm.NewEVM(
        vm.BlockContext{
            CanTransfer: CanTransfer,
            Transfer:    Transfer,
            GetHash:     GetHashFn(ctx, k),
            Coinbase:    k.GetCoinbaseAddress(ctx),
            BlockNumber: big.NewInt(ctx.BlockHeight()),
            Time:        big.NewInt(ctx.BlockTime().Unix()),
            Difficulty:  big.NewInt(0), // PoS
            GasLimit:    k.GetBlockGasLimit(ctx),
        },
        vm.TxContext{
            Origin:   sender,
            GasPrice: tx.GasPrice(),
        },
        stateDB,
        cfg,
    )

    // 3. Apply message
    msg := AsMessage(tx)

    // 4. Execute
    ret, leftoverGas, err := k.ApplyMessage(evm, msg, cfg)

    // 5. Calculate gas used
    gasUsed := msg.Gas() - leftoverGas

    // 6. Refund gas
    k.RefundGas(ctx, msg, leftoverGas)

    return &ExecutionResult{
        Ret:      ret,
        GasUsed:  gasUsed,
        Err:      err,
        Logs:     stateDB.GetLogs(tx.Hash()),
    }, nil
}
```

ApplyMessage

```go
func (k *Keeper) ApplyMessage(
    evm *vm.EVM,
    msg Message,
    cfg *EVMConfig,
) ([]byte, uint64, error) {
    // 1. Check nonce
    if msg.CheckNonce() {
        nonce := evm.StateDB.GetNonce(msg.From())
        if nonce < msg.Nonce() {
            return nil, 0, ErrNonceTooHigh
        } else if nonce > msg.Nonce() {
            return nil, 0, ErrNonceTooLow
        }
    }

    // 2. Buy gas
    mgval := new(big.Int).SetUint64(msg.Gas())
    mgval.Mul(mgval, msg.GasPrice())
    if err := evm.StateDB.SubBalance(msg.From(), mgval); err != nil {
        return nil, 0, err
    }

    // 3. Increment nonce
    evm.StateDB.SetNonce(msg.From(), evm.StateDB.GetNonce(msg.From())+1)

    // 4. Execute
    var (
        ret   []byte
        vmerr error
    )

    if msg.To() == nil {
        // Contract creation
        ret, _, vmerr = evm.Create(
            vm.AccountRef(msg.From()),
            msg.Data(),
            msg.Gas(),
            msg.Value(),
        )
    } else {
        // Contract call
        ret, vmerr = evm.Call(
            vm.AccountRef(msg.From()),
            *msg.To(),
            msg.Data(),
            msg.Gas(),
            msg.Value(),
        )
    }

    // 5. Calculate gas used
    gasUsed := msg.Gas() - evm.StateDB.GetGasUsed()

    return ret, gasUsed, vmerr
}
```

Account 모델

Ethereum vs Cosmos

측면	Ethereum	Cosmos SDK	Cosmos EVM
Account ID	20 bytes address	Bech32 address	Both supported
Nonce	Per account	Per account (sequence)	Mapped
Balance	Single ETH	Multiple denoms	ETH ↔ native token
Code	Embedded	N/A	Stored separately
Storage	Merkle Patricia	IAVL tree	Mapped

Address Conversion

```go
// Convert Ethereum address to Cosmos address
func EthAddressToCosmosAddress(ethAddr common.Address) sdk.AccAddress {
    return sdk.AccAddress(ethAddr.Bytes())
}

// Convert Cosmos address to Ethereum address
func CosmosAddressToEthAddress(cosmosAddr sdk.AccAddress) common.Address {
    return common.BytesToAddress(cosmosAddr.Bytes())
}
```

Account 생성

```go
func (k *Keeper) CreateAccount(ctx sdk.Context, addr common.Address) {
    // 1. Create Cosmos SDK account
    cosmosAddr := sdk.AccAddress(addr.Bytes())
    account := k.accountKeeper.NewAccountWithAddress(ctx, cosmosAddr)
    k.accountKeeper.SetAccount(ctx, account)

    // 2. Initialize EVM state
    stateDB := NewStateDB(ctx, k)
    stateDB.CreateAccount(addr)
}
```

Storage Management

Storage Layout

```
Storage Key = [prefix][address][slot]
            = [0x03][20 bytes][32 bytes]

Example:
  Contract: 0x1234...5678
  Slot: 0x0000...0001
  Key: 0x03 + 0x1234...5678 + 0x0000...0001
```

Storage 읽기/쓰기

```go
// Read storage
func (k *Keeper) GetStorage(
    ctx sdk.Context,
    addr common.Address,
    key common.Hash,
) common.Hash {
    store := ctx.KVStore(k.storeKey)
    bz := store.Get(StorageKey(addr, key))

    if bz == nil {
        return common.Hash{}
    }

    return common.BytesToHash(bz)
}

// Write storage
func (k *Keeper) SetStorage(
    ctx sdk.Context,
    addr common.Address,
    key, value common.Hash,
) {
    store := ctx.KVStore(k.storeKey)

    if value == (common.Hash{}) {
        // Delete if zero
        store.Delete(StorageKey(addr, key))
    } else {
        store.Set(StorageKey(addr, key), value.Bytes())
    }
}
```

Storage Gas

```go
// Gas costs for storage operations
const (
    SstoreSetGas    = 20000  // Set storage (first time)
    SstoreResetGas  = 5000   // Reset storage (update)
    SstoreClearGas  = 5000   // Clear storage (to zero)
    SstoreRefundGas = 15000  // Refund for clearing
)

func (s *StateDB) SetState(addr common.Address, key, value common.Hash) {
    obj := s.GetOrNewStateObject(addr)
    if obj != nil {
        // Journal for revert
        prev := obj.GetState(key)
        s.journal.append(storageChange{
            account:  &addr,
            key:      key,
            prevalue: prev,
        })

        // Set state
        obj.SetState(key, value)

        // Gas accounting
        if prev == (common.Hash{}) && value != (common.Hash{}) {
            // Setting non-zero value
            s.AddGas(SstoreSetGas)
        } else if prev != (common.Hash{}) && value == (common.Hash{}) {
            // Clearing storage
            s.AddGas(SstoreClearGas)
            s.AddRefund(SstoreRefundGas)
        } else {
            // Updating value
            s.AddGas(SstoreResetGas)
        }
    }
}
```

Gas Mechanism

Gas 통합

Cosmos EVM은 Ethereum gas와 Cosmos SDK gas를 통합합니다.

```go
// Gas conversion rate
const (
    // 1 EVM gas = X Cosmos SDK gas
    GasConversionRate = 1000
)

// Convert EVM gas to Cosmos gas
func EVMGasToCosmosGas(evmGas uint64) uint64 {
    return evmGas * GasConversionRate
}

// Convert Cosmos gas to EVM gas
func CosmosGasToEVMGas(cosmosGas uint64) uint64 {
    return cosmosGas / GasConversionRate
}
```

Gas Meter 통합

```go
// keeper/statedb.go
func (s *StateDB) AddGas(amount uint64) {
    // Convert EVM gas to Cosmos gas
    cosmosGas := EVMGasToCosmosGas(amount)

    // Consume Cosmos SDK gas
    s.ctx.GasMeter().ConsumeGas(cosmosGas, "EVM execution")
}

func (s *StateDB) GetGasUsed() uint64 {
    // Get Cosmos gas consumed
    cosmosGas := s.ctx.GasMeter().GasConsumed()

    // Convert to EVM gas
    return CosmosGasToEVMGas(cosmosGas)
}
```

Precompiled Contracts

Precompiled contracts는 EVM에서 Cosmos SDK 기능을 호출할 수 있게 합니다.

표준 Precompiles

```go
var (
    // Standard Ethereum precompiles
    PrecompiledAddresses = map[common.Address]bool{
        common.BytesToAddress([]byte{1}): true, // ecRecover
        common.BytesToAddress([]byte{2}): true, // SHA256
        common.BytesToAddress([]byte{3}): true, // RIPEMD160
        common.BytesToAddress([]byte{4}): true, // Identity
        common.BytesToAddress([]byte{5}): true, // ModExp
        common.BytesToAddress([]byte{6}): true, // ecAdd
        common.BytesToAddress([]byte{7}): true, // ecMul
        common.BytesToAddress([]byte{8}): true, // ecPairing
        common.BytesToAddress([]byte{9}): true, // Blake2F
    }
)
```

Custom Cosmos Precompiles

```go
// Example: Staking precompile
// Address: 0x0000...0800
type StakingPrecompile struct {
    keeper stakingkeeper.Keeper
}

func (p *StakingPrecompile) RequiredGas(input []byte) uint64 {
    return 10000
}

func (p *StakingPrecompile) Run(input []byte) ([]byte, error) {
    // Parse method selector
    if len(input) < 4 {
        return nil, errors.New("invalid input")
    }

    method := input[:4]

    switch string(method) {
    case "dele": // delegate
        return p.delegate(input[4:])
    case "unde": // undelegate
        return p.undelegate(input[4:])
    default:
        return nil, errors.New("unknown method")
    }
}

func (p *StakingPrecompile) delegate(input []byte) ([]byte, error) {
    // Decode input: validator address, amount
    // Call staking keeper
    // Return result
}
```

실전 코드 분석

Journal (Revert 메커니즘)

```go
// types/journal.go
type journal struct {
    entries []journalEntry
    dirties map[common.Address]int
}

type journalEntry interface {
    revert(*StateDB)
    dirtied() *common.Address
}

// Example: balance change
type balanceChange struct {
    account *common.Address
    prev    *big.Int
}

func (ch balanceChange) revert(s *StateDB) {
    s.GetAccount(*ch.account).setBalance(ch.prev)
}

func (ch balanceChange) dirtied() *common.Address {
    return ch.account
}

// Snapshot & Revert
func (s *StateDB) Snapshot() int {
    id := s.nextRevisionId
    s.nextRevisionId++
    s.validRevisions = append(s.validRevisions, revision{
        id:           id,
        journalIndex: s.journal.length(),
    })
    return id
}

func (s *StateDB) RevertToSnapshot(revid int) {
    idx := sort.Search(len(s.validRevisions), func(i int) bool {
        return s.validRevisions[i].id >= revid
    })

    if idx == len(s.validRevisions) || s.validRevisions[idx].id != revid {
        panic("revision not found")
    }

    snapshot := s.validRevisions[idx].journalIndex

    // Revert changes
    s.journal.revert(s, snapshot)
    s.validRevisions = s.validRevisions[:idx]
}
```

참고 자료

공식 문서

- Cosmos EVM GitHub
- go-ethereum EVM
- EVM Opcodes

소스 코드

- StateDB Implementation
- EVM Keeper

다음 단계

1. **Cosmos EVM vs go-ethereum** → 07-cosmos-evm-vs-geth.md
2. **Precompiled Contracts Guide** → 08-precompile-guide.md

작성자: B-Harvest Core Developer Candidate **버전**: 1.0 **최종 수정일**: 2025-01-XX